Unity 2D Platform Runner

Comprehensive Technical Architecture & Project Report

Project Name:	2D Platform Runner (Unity Prototype)	**Engine / Version:**	Unity 2022+ / C#
Repository:	NehaPrajapat0101/2d-platform-runner	**Target Platform:**	PC / Desktop (Windows)
Architecture Pattern:	Component-based + Manager Singletons	**Report Date:**	August 2026

1. Executive Summary

The **2D Platform Runner** is a fully realized 2D action-platformer prototype engineered in Unity with C#. The project demonstrates industry-standard game architecture, including modular player locomotion, dynamic physics & custom gravity scaling, Animator state machine blends, proximity-based enemy artificial intelligence, persistent level unlock tracking via PlayerPrefs, and decoupled enum-keyed audio management.

Core Feature Matrix:

Subsystem	Implementation Details	Status
Player Locomotion	Walk, sprint (Shift), jump (Space), crouch (Ctrl) with speed animation blends	**Completed**
Physics & Gravity	Dynamic gravity scaling (Normal / Jump / Fall) eliminating floaty jumps	**Completed**
Dynamic Colliders	Runtime BoxCollider2D size & offset adjustments per character animation state	**Completed**
Enemy AI System	Distance-based proximity detection, directional sprite flipping & melee attacks	**Completed**
Level Manager	Persistent Singleton manager, PlayerPrefs level unlock/completion tracking	**Completed**
Health & Lives	Dynamic UI heart container with Red-to-Grey heart sprite & runtime animator swap	**Completed**
Audio Manager	Enum-keyed audio engine managing background loops, SFX & one-shot triggers	**Completed**
UI & Pause System	Time.timeScale pause toggle, background blur panel, lobby return & scene reloader	**Completed**

2. Controls & Input Mapping Schema

Hardware input is captured via Unity's Input manager, mapping immediate mechanical reactions to physics bodies and Animator parameter updates.

Key / Binding	Action	Internal Logic & Execution Strategy
`A / D or Arrow Keys`	Horizontal Move	Applies horizontal velocity to Rigidbody2D; updates transform.localScale.x
`Left Shift`	Sprint / Run	Increases base speed from 5 to 8 units/s; sets Animator float parameter Speed=0.3
`Spacebar`	Jump	Applies vertical impulse velocity; checks ground state; adds sprint jump bonus (jumpForce + deltaJump)
`Left / Right Ctrl`	Crouch Toggle	Toggles isCrouching boolean; resizes BoxCollider2D to crouch bounds
`Q Key`	Weapon Switch	Toggles hasGun boolean; updates Animator layer for gun stance
`J Key / Left Click`	Melee Attack	Fires Animator Attack trigger; plays attack audio via AudioManager

3. Technical Architecture & Core Subsystems

3.1 Player Mechanics & Dynamic Physics (PlayerController.cs)

The player avatar is controlled by PlayerController.cs, coordinating locomotion physics, custom gravity scales, invincibility routines, health loss callbacks, key pickup events, and scene reload triggers.

Custom Dynamic Gravity Scale Logic:

To eliminate standard Unity floaty jumps, dynamic gravity scale adjustments are performed in HandleGravity() based on vertical velocity vector:

```
void HandleGravity()
{
    if (rb2D.velocity.y > 0)       // Ascending phase of jump
        rb2D.gravityScale = jumpGravity;
    else if (rb2D.velocity.y < 0)  // Descending / Fall phase
        rb2D.gravityScale = fallGravity;
    else                           // Grounded locomotion
        rb2D.gravityScale = normalGravity;
}
```

Runtime BoxCollider2D Resizing:

To maintain pixel-accurate collision bounds during animations, the character's BoxCollider2D dynamically updates size and offset vectors:

Character State	Collider Size Vector (X, Y)	Collider Offset Vector (X, Y)
Standing Locomotion	standingSize (Cached on Awake)	standingOffset (Cached on Awake)
Crouched Stance	crouchSize (Inspector Configured)	crouchOffset (Inspector Configured)
Airborne Jump	jumpingSize (Inspector Configured)	jumpingOffset (Inspector Configured)

3.2 Enemy Artificial Intelligence (EnemyController.cs)

Enemies utilize real-time distance evaluation to transition between Idle, Patrol, Chase, and Melee Attack states based on player proximity:

Proximity Distance Metric	Calculated Speed	AI Behavior & Animator Parameters
`distanceX > 15f || distanceY > 1f`	0.1 units/s	Idle / Stationary; Animator PlayerDistance=0
`distanceX < 8f && distanceY < 1f`	4.0 units/s	Patrol Approach; Animator PlayerDistance=1
`distanceX < 4f && distanceY < 1f`	6.0 units/s	Aggressive Pursuit; Animator PlayerDistance=2
`distanceX < 1f && distanceY < 1f`	Attack Speed	Melee Attack Triggered (Animator isAttacking=true)

3.3 Level Progression & Persistence (LevelManager.cs)

Level unlock state is saved persistently using Unity's PlayerPrefs key-value database, managed by a persistent LevelManager singleton.

LevelStatus Enum	PlayerPrefs Int	UI Button Color Code	Access Rights & Behavior
Locked	0	Dark Brown (108, 76, 76, 255)	Button disabled; plays error SFX (SoundNames.Faah)
Unlocked	1	Bright Green (85, 255, 0, 255)	Playable level; initiates transition to loading screen
Completed	2	Clean White (255, 255, 255, 255)	Completed level; re-playable scene selection

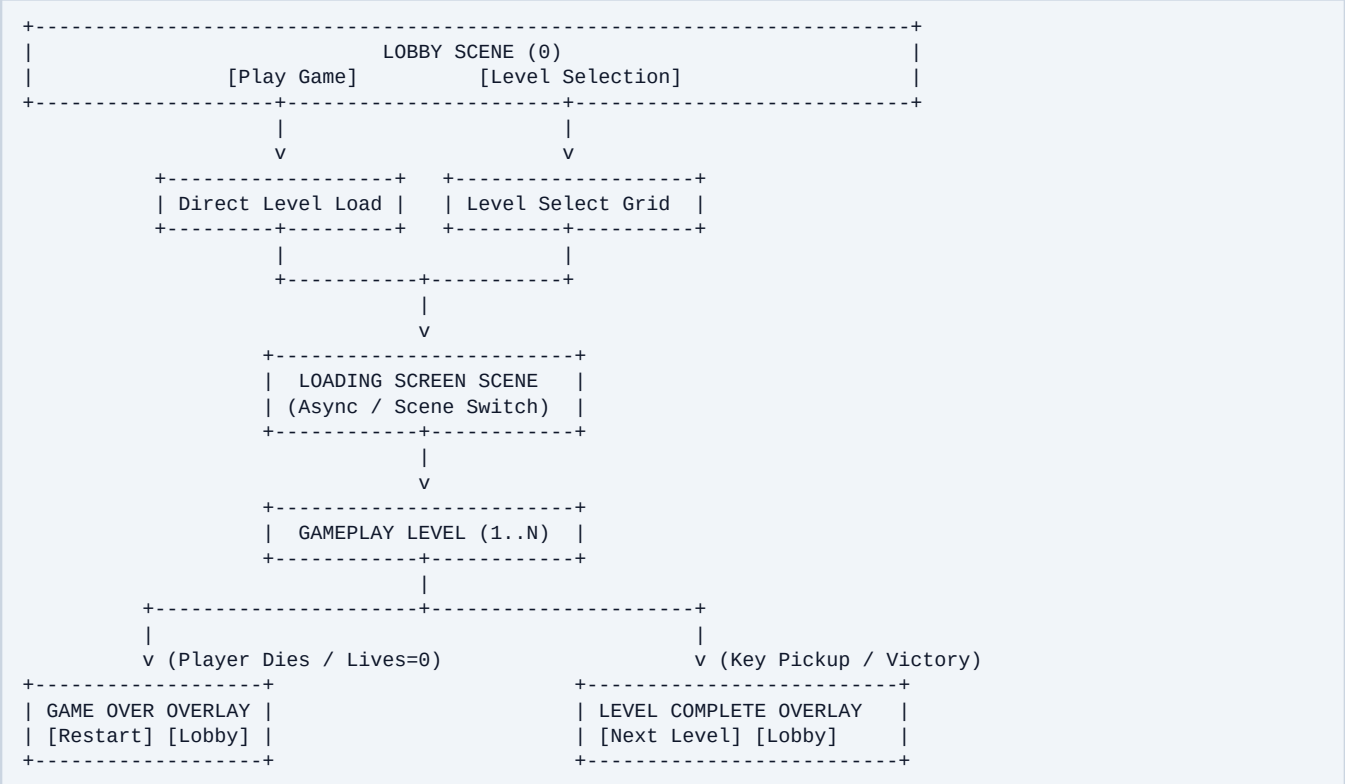
3.4 Audio Architecture (AudioManager.cs & SoundNames.cs)

Audio is fully decoupled using an enum-driven central AudioManager singleton. Each sound clip is wrapped in a Sound object and instantiated with dedicated AudioSource components on game start.

Registered Sound Triggers (SoundNames Enum):
EnvironmentalAmbience, MenuButtonPlay, MenuButtonQuit, PlayerJump, PlayerLand, PlayerWalk, PlayerMeeleAttack, LevelSelection, PlayerDeath, Background, LevelOverUI, KeyPickup, HomePage, LevelSelectionBackground, Faah.

4. Game Scene Flow & Execution Pipeline

The application architecture separates navigation menus, asynchronous loading screens, and gameplay level loops to ensure crisp performance:



5. Codebase Directory & Asset Hierarchy

Project files follow standard Unity asset organization rules, grouping components by responsibility:

```
Platform_run/
■■■ Assets/
■   ■■■ Animations/              # Player & Enemy Animator Controllers + Clips
■   ■■■ Editor/                  # Custom Unity Editor helper utilities
■   ■■■ Game Assets/             # Environment Sprites, Textures, Materials
■   ■■■ Resources/               # Runtime dynamic loadable assets
■   ■■■ Scenes/
■   ■   ■■■ Lobby.unity          # Main menu scene (Build Index 0)
■   ■   ■■■ LoadingScreen.unity  # Intermediate async loading scene
■   ■   ■■■ 1.unity              # Primary Gameplay Level scene
■   ■■■ Scripts/
■   ■   ■■■ Audio/
■   ■   ■   ■■■ AudioManager.cs   # Central audio manager singleton
■   ■   ■   ■■■ Sound.cs          # Audio metadata container struct
■   ■   ■   ■■■ SoundNames.cs     # Enum audio clip key definitions
■   ■   ■■■ LevelS/
■   ■   ■   ■■■ LevelManager.cs   # Persistent progression singleton
■   ■   ■   ■■■ LevelStatus.cs    # Enum (Locked, Unlocked, Completed)
■   ■   ■   ■■■ LevelsLoad.cs     # Level select button status controller
■   ■   ■   ■■■ LoadingBeforeLevel.cs
■   ■   ■■■ PlayerController.cs   # Core player locomotion & dynamic physics
■   ■   ■■■ EnemyController.cs    # Enemy proximity pursuit & combat AI
■   ■   ■■■ HeartController.cs    # Dynamic UI heart health display
■   ■   ■■■ KeyController.cs      # Collectible key trigger script
■   ■   ■■■ ScoreController.cs    # TextMeshPro score tracker
■   ■   ■■■ PauseManager.cs       # Time.timeScale game pause controller
■   ■   ■■■ GameOverController.cs # Game over UI & level reload logic
■   ■■■ Tilemaps/                # 2D Tilemap palettes & environment ground tiles
■■■ ProjectSettings/             # Unity Engine Build & Input Configuration
■■■ README.md                    # Technical overview document
```

6. Quality Assurance & Future Feature Roadmap

Empirical QA Verification Matrix:

• **Physics & Locomotion:** Movement speeds (5 walk / 8 sprint) and variable jump gravity function accurately across framerates.
• **Animation Blend Trees:** Locomotion speed parameter, crouch toggle, weapon equip state, and attack triggers perform smoothly.
• **Invincibility & Damage Handling:** Taking enemy hit triggers invincibility coroutine, reducing UI hearts without double-counting damage.
• **Audio Event Delivery:** Background music, footstep audio, jump SFX, melee swings, and key pickup sounds fire accurately without overlap issues.

Future Feature Development Roadmap:

Feature Feature	Technical Target & Implementation Strategy	Priority Level
Ranged Shooting Mechanics	Instantiate bullet prefabs upon attack trigger when hasGun=true	**High**
Expanded Enemy AI Types	Implement flying enemy AI, patrol waypoints, and ranged enemy projectiles	**High**
Collectibles & Powerups	Add coins, health potions, temporary invincibility stars, speed boosters	Medium
Traps & Obstacles	Ground spikes, moving platforms, falling hazards, collapsible bridges	Medium
Profile Save System	JSON serialization for multi-slot local and cloud save files	Low

End of Technical Project Report — Unity 2D Platform Runner